TMDB → Python → PostgreSQL → Power BI Detailed Step-by-Step Project Instructions

This document walks you through a complete end-to-end pipeline: obtaining a TMDB API key, extracting data with Python, transforming data using pandas, exporting to PostgreSQL, connecting Power BI, and automating the pipeline. Follow each step carefully. The instructions assume you're using Windows for scheduler steps but include notes for Linux/macOS.

1. Prerequisites

• Python 3.8+ installed and available as `python` or `python3`. • PostgreSQL installed and running. • Power BI Desktop installed (for Windows). • Basic familiarity with command line and Python packages. • A free TMDB account to obtain an API key.

2. Create TMDB API Key

1. Open your browser and go to https://www.themoviedb.org/signup and create a free account. 2. After verifying the email and logging in, click your profile → Settings → API. 3. Apply for an API key for personal use. You will get an API key (v3) and a Bearer token (v4). For this project, the v3 API key is sufficient. 4. Save the API key somewhere safe (we'll use a .env file to store it).

3. Create project directory and virtual environment

Open your terminal/PowerShell and run the following commands to create a project folder and virtual environment:

```
mkdir tmdb_pipeline
cd tmdb_pipeline
python -m venv .venv
# Activate the venv:
# Windows (PowerShell): .\.venv\Scripts\Activate.ps1
# Windows (cmd): .\.venv\Scripts\activate.bat
# macOS/Linux: source .venv/bin/activate
```

4. Install required Python packages

Install packages required for extraction, transformation, and loading to PostgreSQL.

```
pip install requests pandas sqlalchemy psycopg2-binary python-dotenv
```

5. Create a .env file to store secrets

Create a file named `.env` in the project root with the following content (replace placeholders):

```
TMDB_API_KEY=your_tmdb_api_key_here
PG_USER=postgres
PG_PASSWORD=your_postgres_password
PG_HOST=localhost
PG_DB=tmdb_db
PG_PORT=5432
```

6. PostgreSQL setup

1. Start PostgreSQL server (instructions depend on your OS). Ensure you know the admin user (often 'postgres') and password. 2. Create a new database for the project. Using psql or a GUI (pgAdmin / Azure Data Studio):

```
-- connect as postgres admin
CREATE DATABASE tmdb_db;
-- optionally create a dedicated user:
CREATE USER tmdb_user WITH PASSWORD 'secure_password';
GRANT ALL PRIVILEGES ON DATABASE tmdb_db TO tmdb_user;
```

Note: If you use a dedicated user, update the .env file values accordingly (PG_USER, PG_PASSWORD).

7. Python script: extract, transform, load

Create a file `tmdb_to_postgres.py` with the following contents. This script performs: extract (TMDB Popular movies), transform (keep selected columns and convert types), and load (to PostgreSQL using SQLAlchemy).

```
import os
import requests
import pandas as pd
from sqlalchemy import create_engine
from dotenv import load_dotenv
load_dotenv()
API_KEY = os.getenv('TMDB_API_KEY')
PG_USER = os.getenv('PG_USER', 'postgres')
PG_PASSWORD = os.getenv('PG_PASSWORD', 'password')
PG_HOST = os.getenv('PG_HOST', 'localhost')
PG_DB = os.getenv('PG_DB', 'tmdb_db')
PG_PORT = os.getenv('PG_PORT', '5432')
def extract_popular_movies(page=1):
    url = f"https://api.themoviedb.org/3/movie/popular?api_key={API_KEY}&language;=en-US&page;={p
age}"
    r = requests.get(url)
    r.raise_for_status()
    return r.json()
def transform_movies(results):
    df = pd.DataFrame(results)
    cols = ['id','title','vote_average','vote_count','popularity','release_date','original_langua
ge']
    df = df.reindex(columns=cols)
    df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
    return df
def get_engine():
    url = f"postgresql+psycopg2://{PG_USER}:{PG_PASSWORD}@{PG_HOST}:{PG_PORT}/{PG_DB}"
    return create_engine(url, connect_args={})
def load_to_postgres(df, table_name='popular_movies'):
    engine = get_engine()
    df.to_sql(table_name, engine, if_exists='replace', index=False)
    print(f"Loaded {len(df)} rows into {table_name}")
if __name__ == '__main__':
    data = extract_popular_movies(page=1)
    df = transform_movies(data['results'])
    load_to_postgres(df)
```

8. Run the script and verify data

1. Activate your virtual environment if not active. 2. Run:

```
python tmdb_to_postgres.py
```

3. Verify inside PostgreSQL using psql / pgAdmin / Azure Data Studio:

```
SELECT count(*) FROM popular_movies;
SELECT * FROM popular_movies LIMIT 5;
```

9. Extend extraction: multiple pages and Netflix availability (optional)

TMDB returns pages of results. To retrieve more records, loop over pages. To check if a movie is available on Netflix in a country, use the `watch/providers` endpoint.

```
# Example: get multiple pages
all_results = []
for p in range(1, 6):
    data = extract_popular_movies(page=p)
    all_results.extend(data['results'])
df = transform_movies(all_results)
# Example: check providers for one movie (country: US)
movie_id = 550
prov = requests.get(f"https://api.themoviedb.org/3/movie/{movie_id}/watch/providers?api_key={API_
KEY}").json()
# Look into prov['results'].get('US') to find 'flatrate' providers like Netflix
```

10. Schema control and explicit table creation (optional, recommended for production)

If you want tight control over column types, create the table manually in PostgreSQL and then use `if_exists='append'` in `to_sql`. Example DDL:

```
CREATE TABLE popular_movies (
    id BIGINT PRIMARY KEY,
    title TEXT,
    vote_average DOUBLE PRECISION,
    vote_count BIGINT,
    popularity DOUBLE PRECISION,
    release_date DATE,
    original_language TEXT
);
```

Then in Python call `df.to_sql('popular_movies', engine, if_exists='append', index=False)` after ensuring that dataframe columns match table columns and types.

11. Power BI: connect to PostgreSQL

1. Open Power BI Desktop → Get Data → PostgreSQL database. 2. Enter server (PG_HOST) and database (PG_DB). Use the appropriate authentication (database user or Windows credentials). 3. Select `popular_movies` table and load. 4. Build visuals (histograms of vote_average, top N by popularity, release year distribution).

Notes: If your PostgreSQL is on localhost and you want scheduled refresh on the Power BI Service, install and configure the On-premises data gateway and configure datasource credentials in the Power BI service.

12. Automation options

Option A: Windows Task Scheduler (simple) 1. Create a batch file `run_pipeline.bat` with content:

```
@echo off
cd /d C:\path\to\tmdb_pipeline
.venv\Scripts\python.exe tmdb_to_postgres.py
```

2. Open Task Scheduler → Create Task → Trigger (Daily/Hourly) → Action: Start a program → Browse to `run_pipeline.bat`. 3. Ensure 'Run whether user is logged on or not' if required; configure credentials accordingly.

Option B: Linux cron 1. Edit crontab with `crontab -e` and add a line to run every hour:

```
0 * * * * /home/youruser/tmdb_pipeline/.venv/bin/python /home/youruser/tmdb_pipeline/tmdb_to_postgres.py >> /home/youruser/tmdb_pipeline/log.txt 2>&1
```

Option C: Airflow (production-grade) • Create a DAG that runs extract→transform→load tasks with retries, logging, and monitoring. Use a PostgresHook / SqlAlchemy operator.

13. Logging, error handling and retries

• Wrap API calls with try/except and use `requests` retry/backoff logic (urllib3 Retry or tenacity library). • Log to a file using Python's `logging` module. • If loading large data, use COPY / COPY_FROM for speed (psycopg2.extras.execute_batch or COPY from CSV).

14. Security best practices

• Never commit `.env` to source control. Add it to `.gitignore`. • Use dedicated DB users with least privileges. • Store production secrets in a secret manager (AWS Secrets Manager, Azure Key Vault) when deploying. • Rotate API keys periodically.

15. Enhancements and next steps

• Incremental loads: store last updated timestamp and only fetch newer items. • Normalized schema: separate genres, providers, and cast into related tables and use foreign keys. • Use parquet files for intermediate storage and faster reloads. • Build a Power BI template with predefined visuals and queries. • Add unit tests for transformation functions and CI/CD pipeline for deployment.

16. Troubleshooting common errors

• `ModuleNotFoundError`: ensure venv is activated and packages installed. • `requests.exceptions.HTTPError`: print response status and body; check API key and rate limits. • `OperationalError` connecting to Postgres: verify host, port, user, password, and that server allows connections (pg_hba.conf).

• `to_sql` performance slow: use `if_exists='append'` with batches, or use psycopg2 COPY for bulk load.

17. Full example: expanded pipeline (save as pipeline.py)

A single orchestrating script that calls functions and logs progress. This mirrors the smaller script earlier but loops pages and handles basic retries.

```
import os, time, logging
import requests
import pandas as pd
from sqlalchemy import create_engine
from dotenv import load_dotenv
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
# setup logging
logging.basicConfig(level=logging.INFO, filename='pipeline.log', filemode='a',
                    format='%(asctime)s %(levelname)s:%(message)s')
load_dotenv()
API_KEY = os.getenv('TMDB_API_KEY')
PG_USER = os.getenv('PG_USER')
```

```
PG_PASSWORD = os.getenv('PG_PASSWORD')
PG_HOST = os.getenv('PG_HOST')
PG_DB = os.getenv('PG_DB')
PG_PORT = os.getenv('PG_PORT', '5432')
def requests_session_with_retries(total_retries=5, backoff=1):
    s = requests.Session()
    retries = Retry(total=total_retries, backoff_factor=backoff,
                    status_forcelist=[429,500,502,503,504])
    s.mount('https://', HTTPAdapter(max_retries=retries))
    return s
def extract_all_pages(max_pages=5):
    s = requests_session_with_retries()
    all_results = []
    for p in range(1, max_pages+1):
        url = f"https://api.themoviedb.org/3/movie/popular?api_key={API_KEY}&language;=en-US&page
;={p}"
        r = s.get(url)
        r.raise_for_status()
        j = r.json()
        all_results.extend(j.get('results', []))
        time.sleep(0.25)
    return all_results
def transform(rows):
    df = pd.DataFrame(rows)
    cols = ['id','title','vote_average','vote_count','popularity','release_date','original_langua
ge']
    df = df.reindex(columns=cols)
    df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
    return df
def get_engine():
    url = f"postgresql+psycopg2://{PG_USER}:{PG_PASSWORD}@{PG_HOST}:{PG_PORT}/{PG_DB}"
    return create_engine(url)
def load(df, table_name='popular_movies'):
    engine = get_engine()
    df.to_sql(table_name, engine, if_exists='replace', index=False, method='multi')
    logging.info(f"Loaded {len(df)} rows")
if __name__ == '__main__':
    try:
        rows = extract_all_pages(max_pages=5)
        df = transform(rows)
        load(df)
    except Exception as e:
        logging.exception('Pipeline failed')
```

18. Deliverables checklist

• TMDB API key • Project folder with scripts and .env • PostgreSQL database with `popular_movies` table • Power BI report connected to the DB • Automation configured (Task Scheduler / cron / Airflow) • Logging and simple retry logic

19. References and useful links

• TMDB API docs: https://developer.themoviedb.org/ • Requests retries: urllib3 Retry docs • SQLAlchemy docs: https://docs.sqlalchemy.org/ • Psycopg2 docs: https://www.psycopg.org/ • Power BI: https://powerbi.microsoft.com/

If you want, I can also: • Add a ready-to-run GitHub repo with the scripts. • Add a Power BI template (.pbit) with the visuals pre-built. • Customize the pipeline to include Netflix availability and normalize the schema. Good luck! Run the scripts, and if you hit any errors paste the error message and I will help you debug.